STRIPE AI INFRASTRUCTURE • CERTIFICATION BLOC 2 (RNCP 41993)

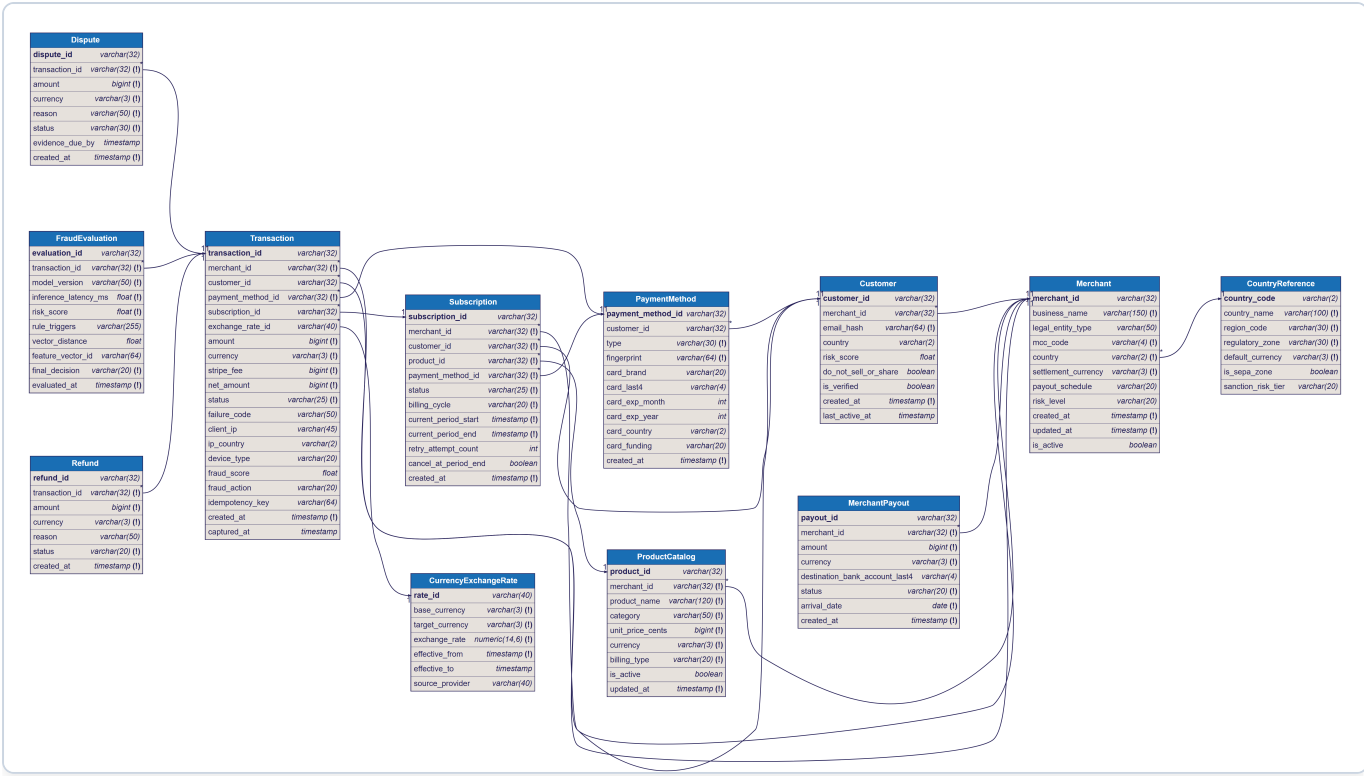
Modèle Relationnel OLTP — Transactions, Abonnements & Garanties ACID (3NF)

Le système transactionnel de Stripe est le cœur battant de l'activité économique de ses marchands. Il doit concilier des contraintes d'une rigueur absolue :

- **Intégrité financière sans compromis :** Chaque centime doit être imputé avec certitude (zéro perte d'argent, zéro double débit), couvrant les paiements unitaires, les **abonnements récurrents**, les **remboursements**, les **litiges (chargebacks)** et les **virements marchands**.
- **Haute cadence d'écriture :** Capacité à absorber des milliers d'écritures concurrentes par seconde lors des pics (ex : Black Friday, lancements de billetterie).
- **Normalisation en 3e Forme Normale (3NF) :** Élimination de toute anomalie de mise à jour, d'insertion ou de suppression, avec séparation stricte des **données de référence** (pays, taux de change, catalogue produits).
- **Conformité stricte PCI-DSS v4.0, RGPD & CCPA :** Zéro stockage de numéro de carte (PAN) en clair ; utilisation exclusive de tokens de paiement (`payment_method_id`), pseudonymisation des emails et gestion du consentement *Opt-Out* CCPA.

2. Diagramme Entité-Association OLTP (ERD)

> **Modèle Relationnel Normalisé (3NF - ACID)** modélisé en **DBML** selon les standards industriels de modélisation financière et exporté en haute résolution.
> *Fichier source de modélisation DBML :* `stripe_oltp.dbml` (inclus dans le dossier livrable)


Dispute
dispute_id varchar(32)
transaction_id varchar(32) (f)
amount bigint (f)
currency varchar(3) (f)
reason varchar(50) (f)
status varchar(30) (f)
evidence_due_by timestamp
created_at timestamp (f)
FraudEvaluation
evaluation_id varchar(32)
transaction_id varchar(32) (f)
model_version varchar(50) (f)
inference_latency_ms float (f)
risk_score float (f)
rule_triggers varchar(255)
vector_distance float
feature_vector_id varchar(64)
final_decision varchar(20) (f)
evaluated_at timestamp (f)
Refund
refund_id varchar(32)
transaction_id varchar(32) (f)
amount bigint (f)
currency varchar(3) (f)
reason varchar(50)
status varchar(20) (f)
created_at timestamp (f)
Transaction
transaction_id varchar(32)
merchant_id varchar(32) (f)
customer_id varchar(32)
payment_method_id varchar(32) (f)
subscription_id varchar(32)
exchange_rate_id varchar(40)
amount bigint (f)
currency varchar(3) (f)
stripe_fee bigint (f)
net_amount bigint (f)
status varchar(25) (f)
failure_code varchar(50)
client_ip varchar(45)
ip_country varchar(2)
device_type varchar(20)
fraud_score float
fraud_action varchar(20)
idempotency_key varchar(64)
created_at timestamp (f)
captured_at timestamp
Subscription
subscription_id varchar(32)
merchant_id varchar(32) (f)
customer_id varchar(32) (f)
product_id varchar(32) (f)
payment_method_id varchar(32) (f)
status varchar(25) (f)
billing_cycle varchar(20) (f)
current_period_start timestamp (f)
current_period_end timestamp (f)
retry_attempt_count int
cancel_at_period_end boolean
created_at timestamp (f)
CurrencyExchangeRate
rate_id varchar(40)
base_currency varchar(3) (f)
target_currency varchar(3) (f)
exchange_rate numeric(14,6) (f)
effective_from timestamp (f)
effective_to timestamp
source_provider varchar(40)
PaymentMethod
payment_method_id varchar(32)
customer_id varchar(32)
type varchar(30) (f)
fingerprint varchar(64) (f)
card_brand varchar(20)
card_last4 varchar(4)
card_exp_month int
card_exp_year int
card_country varchar(2)
card_funding varchar(20)
created_at timestamp (f)
ProductCatalog
product_id varchar(32)
merchant_id varchar(32) (f)
product_name varchar(120) (f)
category varchar(50) (f)
unit_price_cents bigint (f)
currency varchar(3) (f)
billing_type varchar(20) (f)
is_active boolean
updated_at timestamp (f)
Customer
customer_id varchar(32)
merchant_id varchar(32)
email_hash varchar(64) (f)
country varchar(2)
risk_score float
do_not_sell_or_share boolean
is_verified boolean
created_at timestamp (f)
last_active_at timestamp
MerchantPayout
payout_id varchar(32)
merchant_id varchar(32) (f)
amount bigint (f)
currency varchar(3) (f)
destination_bank_account_last4 varchar(4)
status varchar(20) (f)
arrival_date date (f)
created_at timestamp (f)
Merchant
merchant_id varchar(32)
business_name varchar(150) (f)
legal_entity_type varchar(50)
mcc_code varchar(4) (f)
country varchar(2) (f)
settlement_currency varchar(3) (f)
payout_schedule varchar(20)
risk_level varchar(20)
created_at timestamp (f)
updated_at timestamp (f)
is_active boolean
CountryReference
country_code varchar(2)
country_name varchar(100) (f)
region_code varchar(30) (f)
regulatory_zone varchar(30) (f)
default_currency varchar(3) (f)
is_sepa_zone boolean
sanction_risk_tier varchar(20)

3. Dictionnaire Détaillé des Entités Transactionnelles & Référentiels

3.1. Cœur Financier et Paiements

- `Merchant` **(Marchand) :** Titulaire du compte Stripe Connect ou Checkout. Porte le `mcc_code` (*Merchant Category Code*), la devise de règlement (`settlement_currency`), le niveau de risque et référence `CountryReference.country_code` .
- `Customer` **(Client Final) :** Acheteur final pseudonymisé. L'adresse email est stockée exclusivement via son empreinte `email_hash` (SHA-256 avec sel). Intègre le drapeau booléen `do_not_sell_or_share` pour respecter immédiatement les requêtes *Opt-Out* du **CCPA** californien.
- `PaymentMethod` **(Moyen de Paiement Tokenisé) :** Représente carte bancaire, SEPA, Apple Pay ou Google Pay. Le numéro complet (PAN) et le CVV restent isolés dans le *Tokenization Vault* CloudHSM ; seule l'infrastructure OLTP manipule le jeton non réversible `payment_method_id` , les `card_last4` et le `fingerprint` cryptographique de détection multi-marchands.
- `Transaction` **(Table Maîtresse d'Encaissement) :** Enregistrement unitaire d'un paiement. Tous les montants (`amount` , `stripe_fee` , `net_amount`) sont typés en entiers `BIGINT` **en centimes** (éliminant les imprécisions de virgule flottante). Porte la colonne `idempotency_key` (index unique) contre les doubles débits, ainsi que les clés optionnelles vers `Subscription` et `CurrencyExchangeRate` .
- `FraudEvaluation` **(Audit d'Inférence Radar) :** Trace légale et MLOps de la décision prise en ligne (< 100 ms) : version exacte du modèle (`model_version`), latence mesurée, score, distance vectorielle et motifs déclencheurs.

3.2. Opérations Post-Paiement & Abonnements Récurrents

- `Subscription` **(Gestion des Abonnements) :** Gère les paiements récurrents SaaS/médias liés à `Merchant` , `Customer` , `ProductCatalog` et `PaymentMethod` . Suit le statut (`trialing` , `active` , `past_due` , `canceled`), les fenêtres de période (`current_period_start/end`) et le compteur `retry_attempt_count` piloté par le modèle **Smart Retries** pour minimiser l'attrition involontaire.
- `Refund` **(Remboursements) :** Enregistre les remboursements totaux ou partiels rattachés à une `Transaction` (`duplicate` , `fraudulent` , `requested_by_customer`).
- `Dispute` **(Contestations / Chargebacks) :** Consigne les litiges bancaires ouverts par les banques émettrices avec suivi du statut probatoire (`needs_response` , `won` , `lost`) et date limite (`evidence_due_by`).
- `MerchantPayout` **(Virements Marchands) :** Historise les décaissements automatiques (quotidiens, hebdomadaires ou mensuels) vers les comptes bancaires des marchands.

3.3. Données de Référence (Master Data)

- `CountryReference` **:** Table de référence ISO 3166-1 des pays et régions (`EMEA` , `NAMER` , `APAC` , `LATAM`), portant la zone réglementaire (`RGPD_EEA` , `CCPA_US` , `UK_GDPR`), l'appartenance SEPA et le niveau de risque géopolitique/embargo.
- `CurrencyExchangeRate` **:** Historique horodaté des taux de conversion interbancaires (`effective_from` / `effective_to`), garantissant l'auditabilité comptable exacte du taux appliqué à l'instant `t` sur les paiements multi-devises.

- `ProductCatalog` : Référentiel des produits et plans d'abonnement définis par chaque marchand (`one_time` ou `recurring` , catégorie, tarif catalogue).

4. Respect Rigoureux des Propriétés ACID & Transactions Multi-Entités

Propriété ACID	Problème Potentiel en Paiement	Implémentation Architecturale sur Aurora PostgreSQL
Atomicité (A)	Débit client validé sans imputation du solde marchand ni création d'abonnement.	Utilisation de blocs transactionnels stricts (`BEGIN ... COMMIT / ROLLBACK`). L'écriture de `Transaction` , l'audit `FraudEvaluation` , et la mise à jour d'état de `Subscription` s'exécutent de manière indivisible. Pour les workflows impliquant le Vault CloudHSM ou des systèmes externes, un pattern **Transactional Outbox / Saga compensatoire** garantit la cohérence atomique.
Cohérence (C)	Solde négatif, montant nul ou devise invalide sur un remboursement.	Contraintes natives SQL (`CHECK (amount > 0)`), intégrité référentielle stricte (`FOREIGN KEY REFERENCES`) vers les tables de référence (`CountryReference` , `CurrencyExchangeRate`).
Isolation (I)	Deux requêtes concurrentes débitent simultanément le même compte (Race Condition).	Niveau d'isolation par défaut **Read Committed** couplé à l'index unique `idempotency_key` et **Snapshot Isolation / `SELECT ... FOR UPDATE`** lors des écritures sur les virements `MerchantPayout` et remboursements cumulés.
Durabilité (D)	Crash matériel d'un nœud juste après l'acquittement client.	Stockage distribué Aurora répliquant de manière synchrone chaque journal *redo log* sur **6 nœuds répartis sur 3 Zones de Disponibilité** avec validation au Quorum **4/6** avant tout acquittement client.

5. Stratégie d'Indexation et Performance

1. `CREATE UNIQUE INDEX idx_txns_idempotency ON transactions (idempotency_key);`
   Recherche en $O(1)$ lors de chaque appel API pour rejeter instantanément les doublons.
2. `CREATE INDEX idx_txns_merchant_created ON transactions (merchant_id, created_at DESC);`
   Optimise les requêtes de consultation du dashboard marchand (derniers paiements).
3. `CREATE INDEX idx_pm_fingerprint ON payment_methods (fingerprint);`
   Permet au moteur de fraude de vérifier en moins de 2 ms l'historique d'utilisation d'une carte sur la plateforme.
4. `CREATE INDEX idx_sub_renewal ON subscriptions (status, current_period_end) WHERE status IN ('active', 'past_due');`
   Index partiel optimisant la sélection des abonnements à renouveler ou à relancer par Smart Retries.

6. Réplication Temps Réel, Basculement (Failover) & Reprise après Sinistre (DRP)

Pour satisfaire une disponibilité de **99,999%** sur des charges financières critiques :

1. **Réplication Synchrone Intra-Région (3 AZs — `eu-west-3` Paris) :**
   Le cluster **Amazon Aurora PostgreSQL** dissocie calcul et stockage. Le volume de stockage partagé

écrit simultanément sur 6 copies (2 par AZ). En lecture, jusqu'à 15 *Aurora Read Replicas* partagent ce même stockage sans latence de réplication physique significative (< 10 ms).

2. **Basculement Automatique (*Automatic Failover*) :**
   En cas de panne du nœud maître d'écriture (*Writer*), Aurora promeut automatiquement le *Reader Replica* prioritaire en moins de **30 secondes (`RTO < 30s`)** via bascule DNS transparente du endpoint cluster, avec **perte de données strictement nulle (`RPO = 0`)**.
3. **Plan de Reprise après Sinistre Multi-Région (*Aurora Global Database*) :**
   Afin de couvrir l'éventualité d'une indisponibilité régionale majeure à Paris (`eu-west-3`), un cluster secondaire passif est maintenu en réplication asynchrone matérielle dédiée vers l'Irlande (`eu-west-1`) ou Francfort (`eu-central-1`) avec une latence typique inférieure à **1 seconde (`RPO < 5s` , `RTO < 5 min`)**, complété par des sauvegardes continues immuables (*Point-in-Time Recovery* sur 35 jours).